# CSE 320 — Computer Networks

## Programming Assignment

*TCP Congestion Control Algorithms Comparison*

### 1. Assignment Overview

In this assignment, students will implement and compare several TCP congestion control algorithms in C. The goal is to understand how congestion is detected, how the congestion window evolves, and how different algorithms respond to packet loss and network congestion.

#### a. Students must simulate or implement one of the following algorithms ([student1 id + student2 id + student3 id] mod 3):

0. TCP Tahoe 1. TCP Reno 2. TCP NewReno

**Ex student1 id= 20180808003**

**student2 id= 20250808004**

**(20180808003 + 20250808004) % 3 = 2 -> TCP NewReno**

#### b. Each algorithm must demonstrate:

- Slow Start
- Congestion Avoidance
- Fast Retransmit
- Fast Recovery (where applicable)

### 2. System Requirements

Your implementation must satisfy the following:

- A single executable must simulate packet transmission over TCP.
- Students must model sender-side congestion window (cwnd) behavior.
- Packet loss events must be configurable.
- Timeout-based loss and triple-duplicate ACK scenarios must be demonstrated
- The program must print congestion window evolution step-by-step.

## 3. Assignment Tasks

### Task 1 — Define Core Data Structures

Create structures for sender state, congestion window, slow-start threshold (ssthresh), ACK counter, and packet events.

### Task 2 — Implement The algo

#### *a. TCP Tahoe*

Implement Slow Start, Congestion Avoidance, and timeout handling. On packet loss, Tahoe must reset cwnd to 1 MSS.

#### *b. TCP Reno*

Similar to Tahoe with Fast Retransmit and Fast Recovery. On triple duplicate ACKs, Reno must avoid full restart.

#### *c. Implement TCP NewReno*

Improved version of Reno with partial ACK handling during Fast Recovery.

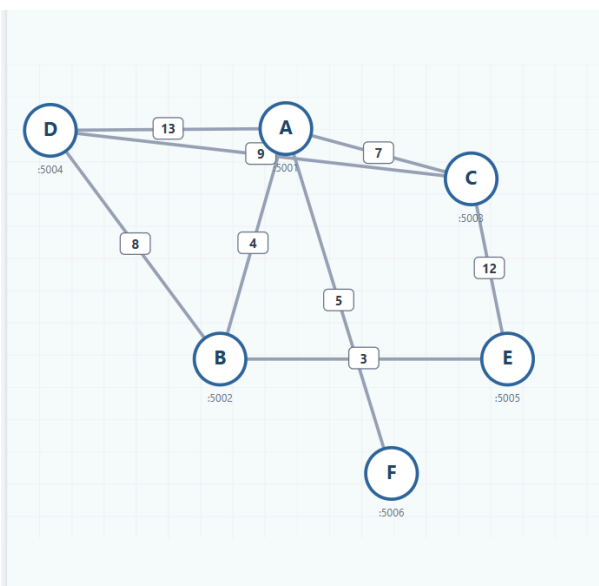
### Task 3 — Experimental Analysis

Run multiple test cases with different packet loss patterns and compare performance using cwnd growth, throughput, and recovery speed.

## 4. Input and Output Expectations

Use the following six-node topology for testing:

Nodes: A, B, C, D, E, F




Ports: A=5001, B=5002, C=5003, D=5004, E=5005, F=5006

Example Configuration Files: A.conf, B.conf, …F.conf (6 files)

You should be able to run them in six separate terminals:

```
./node A.conf
./node B.conf
./node C.conf
./node D.conf
./node E.conf
./node F.conf
```

Expected routing path for node A

```
Routing table for node A

Destination   Next Hop   Cost   Path

A             -          0      A

B             B          4      A -> B

C             C          7      A -> C

D             B          12     A -> B -> D

E             B          7      A -> B -> E

F             F          5      A -> F
```

Important:

- A to D should NOT use A -> D directly.
- A -> D direct cost = 13
- A -> B -> D cost = 4 + 8 = 12
- So next hop from A to D must be B.

From Node A console /terminal tab:

```
send D hello_from_A_to_D
```

Expected forwarding:

```
[A] Destination D, next hop B

[B] Forwarding message from A to D, next hop D

[D] Received message from A: hello_from_A_to_D
```

From Node F console /terminal tab:

```
[A] Destination D, next hop B
```

Expected forwarding:

```
[F] Destination E, next hop A

[A] Forwarding message from F to E, next hop B

[B] Forwarding message from F to E, next hop E

[E] Received message from F: hello_from_F_to_E
```

Students must clearly show how each algorithm behaves differently under the same conditions.

## 5. Submission Requirements

Submit source code, README, and a demonstration video.

The demonstration video must show:

1. Execution of at least two algorithms

2. Congestion window evolution

3. Timeout loss scenario

4. Triple duplicate ACK scenario

5. Brief spoken explanation of algorithm differences

7. Example Testing with Sample Topology